

Getting started

Logging on to ARCHER2

You should already have an account on ARCHER2, and should be able to log on to it using

```
ssh -X accountname@login.archer2.ac.uk
```

(replacing accountname with the name of your account on the system) or with the SSH client of your choice (`-X` ensures that graphics are routed back to your desktop). Once you have successfully logged in you will be presented with an interactive command prompt.

For more detailed instructions on connecting to ARCHER2, or on how to run commands, please see the Appendix.

Download and extract the exercise files

Firstly, change directory to make sure you are on the `/work` filesystem on ARCHER2.

```
cd /work/ta177/ta177/accountname/
```

where ta177 is the name of the project being used for this course. `/work` is a high performance parallel file system that can be accessed by both the frontend and compute nodes. **All jobs on ARCHER2 should be run from the `/work` filesystem.** ARCHER2 compute nodes cannot access the `/home` filesystem at all: any jobs attempting to use `/home` will fail with an error.

Use the following commands (on ARCHER2) to get the exercise files archive from the web and unpack it:

```
cp /work/z19/shared/SN0.tar .
tar xvf SN0.tar
```

Exercise 1: Placement

The code for this exercise is in `Archer20pt/Placement`. The aim of this exercise is to investigate the performance variation across the cores on an ARCHER2 node. Currently the code is setup to run the STREAMs benchmark with 16 cores, all on a single chiplet on the ARCHER2 system. The batch script will run `xthi`, a program that will print out the placement and binding information for the current configuration, and then the STREAMs benchmark. You can build the applications using the command:

```
make
```

You can submit the application to a reservation using the command:

```
sbatch --reservation=XXX run_streams.sh
```

XXX should be replaced by the reservation name given to you at the course. You will also need to specify

```
#SBATCH --qos=reservation
```

in the batch script.

Run the batch script as is, then edit it to run across NUMA regions rather than withing a single NUMA region. You can do this by changing `srun -c 1` to `srun -c 8`. Rerun the application, what is the performance difference? Now alter the batch script to run 32 processes rather than 16 processes. You will also need to alter the `-c 8` flag to `-c 4` to change the process binding. What performance difference do you see? Try for other ranges of process counts up to 128.

Exercise 2: CrayPat

The code for this exercise is in `Archer20pt/VH1`. Please follow the separate instructions in `CrayPAT-intro.pdf`.

Exercise 3: Compiler Optimisation

The code for this exercise is in `Archer20pt/CompilerOpt`. Please follow the separate instruction in `compilerexercise.p`

Exercise 4: Optimisation

The `Archer2Opt/MD/*` directory, where `*` is either `C` or `Fortran`, contains a sequential implementation of a molecular dynamics simulation which has been deliberately written to have poor performance. Use profiling and compiler listings to look for the performance problems. Once you have exhausted the compiler's ability to optimise, try some code modifications.

Exercise 5: Bandwidth and NUMA

The example code can be found in `Archer2Opt/Stream/*`. This is the well-known STREAM benchmark for measuring memory bandwidth. Use the Makefile to compile the code, and run it using different numbers of threads using the supplied batch script.

Does the bandwidth scale linearly with processors? Now try removing the OpenMP loop directive from the initialisation of the arrays. How does the performance change? You can also try using the “wrong” schedule for the loop, or selecting different sets of cores to run on.

Extra exercise

Try reducing the array size `N` by a factor of 100 or 1000 (and increase the repetition count `NTIMES` by the same amount).

Exercise 6: Cache blocking

The `Archer2Opt/Matmul/*` directory contains a simple matrix multiplication code. Try implementing cache blocking by hand, and see the effect on the performance (stick to `-O0` to stop the compiler doing its own optimisation!).

Exercise 7: OpenMP

The `Archer2Opt/MolDyn/*` directory contains a (not very efficient) OpenMP parallel version of a simple molecular dynamics code (not the same as in Exercise 2!). Run the code using the script supplied with the script supplied to measure the performance on 1, 2, 4, 8 and 16 threads.

Try to identify the performance bottlenecks and fix them! You can try using `pat_build -g omp` to instrument the OpenMP runtime library.